From Generation to Verification: A Structured Audit Framework for Reliable AI Mathematical Reasoning

October 1, 2026

Abstract

Large language models (LLMs) have demonstrated impressive capabilities in mathematical reasoning, yet their outputs remain prone to subtle errors—from incorrect intermediate steps to plausible-sounding but false conclusions. We propose a *structured audit framework* that decomposes AI-assisted mathematical reasoning into three layers: **Generation**, **Semantic Reduction**, and **Verification**. Our framework introduces a typed intermediate representation (IR) as the bridge between informal natural-language reasoning and formal verification, enabling systematic reliability assessment. We apply this framework to analyze five representative AI4Math systems (AlphaGeometry, AlphaProof, FunSearch, Ramanujan Machine, and AlphaTensor), identifying specific failure modes and mitigation strategies at each layer. We further propose an operational *reliability checklist* for practitioners. Our analysis reveals that the Semantic Reduction layer is the primary bottleneck for end-to-end reliability, and that bidirectional consistency between IR and natural language is a necessary but insufficient condition for trustworthy AI mathematics.

Keywords: AI for Mathematics Reliability Formal Verification Semantic Reduction Large Language Models

1 Introduction

Artificial intelligence has increasingly penetrated mathematical research, from guiding human intuition in knot theory [4] to discovering new matrix multiplication algorithms [7] and solving olympiad-level geometry [14]. The emergence of large language models (LLMs)—built on the Transformer architecture [15]—has accelerated this trend, with systems like GPT-4 and Claude demonstrating the ability to generate mathematical proofs, solve competition problems, and even propose novel conjectures [11].

However, this progress comes with a critical challenge: *reliability*. AI-generated mathematical reasoning often appears correct on the surface but contains subtle errors—incorrect logical steps, unstated assumptions, or conclusions that do not follow from the premises. Unlike empirical sciences where predictions can be tested against physical reality, mathematical correctness requires *formal verification*: every step must follow from axioms by rules of inference.

The academic community has responded with various reliability frameworks: search-and-verify pipelines [3], formal proof assistants like Lean [6] and Coq, and tool-augmented reasoning [8, 12]. Yet these approaches often address reliability at a single layer—either improving generation quality or adding post-hoc verification—without a systematic framework for understanding where and why failures occur.

Our contribution. We propose a three-layer audit framework that decomposes AI mathematical reasoning into:

1. **Generation Layer**: produces natural-language reasoning (e.g., chain-of-thought);
2. **Semantic Reduction Layer**: translates informal reasoning into structured intermediate representations (typed IR, proof objects, or executable programs);
3. **Verification Layer**: checks correctness via formal methods, symbolic computation, or constraint satisfaction.

We argue that the *Semantic Reduction Layer* is the primary bottleneck for end-to-end reliability. Without stable reduction from natural language to verifiable artifacts, verification has no anchor, and hallucinations persist.

We apply this framework to analyze five representative AI4Math systems, identifying specific failure modes and proposing an operational reliability checklist for practitioners.

2 Background

2.1 AI for Mathematics: A Brief Survey

The application of AI to mathematics spans several paradigms [1, 9]:

- **AI-guided discovery**: Using machine learning to identify patterns and guide human intuition, as demonstrated by DeepMind's work on knot theory and matrix multiplication [4, 7].
- **Automated theorem proving**: Combining neural networks with symbolic reasoning to prove mathematical theorems, exemplified by AlphaGeometry [14] and AlphaProof.
- **Conjecture generation**: Using AI to propose new mathematical conjectures, as seen in the Ramanujan Machine [10] and FunSearch [11].
- **Mathematical problem solving**: Applying LLMs to solve competition-style problems (GSM8K, MATH benchmark) via chain-of-thought [16] and program-aided reasoning [8].

2.2 The Reliability Challenge

Despite these advances, reliability remains a fundamental concern. LLMs are probabilistic text generators, not logical reasoners [2]. They can produce outputs that are:

- **Syntactically correct but semantically wrong**: The proof follows valid formatting but contains logical gaps.
- **Locally correct but globally inconsistent**: Individual steps are valid, but the conclusion does not follow from the premises.
- **Plausible but unverifiable**: The reasoning appears sound but cannot be checked by any known formal method.

These failure modes are particularly dangerous in mathematics, where a single incorrect step invalidates the entire argument.

3 The Three-Layer Audit Framework

We propose decomposing AI-assisted mathematical reasoning into three layers, each with distinct responsibilities and failure modes.

3.1 Generation Layer

The Generation Layer produces natural-language mathematical reasoning. This includes:

- Chain-of-thought (CoT) prompting [16]
- Program-of-thought (PoT) generation [8]
- Tool-augmented reasoning [12]

Characteristics:

- *Probabilistic*: Outputs are sampled from a distribution, not deterministically derived.
- *Untyped*: No formal type system constrains the reasoning.
- *Informal*: Expressed in natural language, which is inherently ambiguous.

Failure modes:

- **Hallucination**: Inventing false statements or non-existent theorems.
- **Logical gaps**: Skipping steps or making unjustified leaps.
- **Inconsistency**: Contradicting earlier statements within the same reasoning chain.

3.2 Semantic Reduction Layer

The Semantic Reduction Layer translates informal natural-language reasoning into structured, machine-checkable artifacts. This is the *critical bottleneck* in our framework.

Target artifacts include:

- **Typed intermediate representations (IR)**: Abstract syntax trees (ASTs) with type annotations.
- **Proof objects**: Formal proof terms in systems like Lean [6] or Coq.
- **Executable programs**: Code that can be run to verify computational claims [8].
- **Constraint systems**: Sets of equations or inequalities for SMT solvers [5].

Theoretical grounding: The Semantic Reduction Layer is motivated by the *Curry-Howard correspondence*, which establishes a deep connection between proofs and programs. Under this correspondence, a mathematical proof can be viewed as a program, and a theorem as a type. The reduction layer thus transforms informal proofs into typed programs that can be checked by a compiler.

Challenges:

- **Ambiguity**: Natural language is inherently ambiguous; multiple formal interpretations may exist.
- **Missing typing**: Informal reasoning often omits type information that must be inferred.
- **Non-invertibility**: The mapping from natural language to IR is many-to-one; information is lost in translation.

Failure modes:

- **Reduction failure**: Unable to produce a valid IR from the natural-language input.
- **Semantic drift**: The IR does not faithfully represent the intended meaning.
- **Type errors**: The IR is ill-typed, indicating a logical inconsistency.

3.3 Verification Layer

The Verification Layer checks the correctness of the structured artifacts produced by the Semantic Reduction Layer.

Verification methods include:

- **Formal proof checking**: Using proof assistants like Lean [6], Coq, or Agda to verify proof objects.
- **Symbolic computation**: Using computer algebra systems to verify algebraic identities.
- **SMT solving**: Using satisfiability modulo theories solvers [5] to check constraint systems.
- **Model checking**: Verifying properties of finite-state systems.

Characteristics:

- *Deterministic*: Verification is a yes/no decision, not probabilistic.
- *Sound*: If verification succeeds, the artifact is correct (modulo the soundness of the verification tool).
- *Incomplete*: Verification may fail even for correct artifacts (e.g., due to undecidability).

Failure modes:

- **Verification failure**: The artifact is incorrect or ill-formed.
- **Verification timeout**: The verification process does not terminate within resource bounds.
- **False negatives**: The artifact is correct but the verifier cannot confirm it.

4 Case Studies: Auditing AI4Math Systems

We apply our three-layer framework to analyze five representative AI4Math systems. For each system, we identify the specific mechanisms at each layer, assess reliability, and analyze failure modes.

4.1 AlphaGeometry: Neuro-Symbolic Geometry Proving

AlphaGeometry [14] is a neuro-symbolic system that solved IMO 2022 Problem 2 (a geometry proof) under competition conditions. It represents a paradigm where neural generation is tightly constrained by symbolic verification.

Generation Layer: A Transformer-based language model trained on 100 million synthetic geometry problems predicts *auxiliary objects*—additional points, lines, or circles that are not given in the problem statement but are necessary for the proof. The model outputs a ranked list of candidate constructions, each represented as a set of geometric constraints (e.g., "point P lies on line AB and circle C").

Semantic Reduction Layer: Each candidate construction is encoded as a system of polynomial equations over the coordinates of geometric objects. The reduction is *exact*: geometric constraints (incidence, parallelism, perpendicularity) map directly to algebraic equations. The reduction layer uses a *symbolic deduction engine* with 200+ deduction rules (e.g., if two angles are equal and two sides are equal, then the triangles are congruent by SAS).

Verification Layer: The deduction engine performs *breadth-first search* over the space of derivable statements, starting from the given constraints and the auxiliary construction. If a proof path exists, BFS is guaranteed to find it (completeness within the rule set). The verification is *exhaustive* and *deterministic*.

Reliability assessment:

- **Strength**: The generation layer is restricted to a *narrow domain* (auxiliary object construction), where the neural network's suggestions can be exhaustively verified. The semantic reduction is *lossless* (geometric constraints $\leftrightarrow$ polynomial equations). The verification is *complete* within the rule set.

- **Weakness**: The system cannot handle problems requiring creative insights beyond the 200+ deduction rules. The neural network may propose constructions that are geometrically invalid (e.g., a point that does not exist), leading to reduction failure.

- **Failure mode**: *Generation-layer hallucination*—the neural network proposes an auxiliary object that is geometrically impossible, causing the deduction engine to fail.

Audit conclusion: AlphaGeometry achieves high reliability ($> 90\%$ on IMO geometry problems) by *constraining the generation layer* and *making the semantic reduction exact*. The system exemplifies Pattern 1 (domain restriction improves reliability).

4.2 AlphaProof: Formal Reasoning with Reinforcement Learning

AlphaProof combines a language model (based on Qwen 2.5) with the Lean 4 theorem prover [6], using reinforcement learning to guide proof search. It achieved a silver-medal level at IMO 2024.

Generation Layer: The language model generates *proof sketches* in a mixture of natural language and Lean 4 code. The model is trained via reinforcement learning, with the reward signal coming from Lean 4's type checker (i.e., whether the generated Lean code type-checks as a valid proof).

Semantic Reduction Layer: The proof sketches are translated into *Lean 4 proof terms*. This is the most challenging layer: the language model must resolve ambiguities (e.g., which induction hypothesis to apply), infer missing type information (e.g., the type of a variable), and produce syntactically valid Lean 4 code. The reduction is *lossy*: some natural-language reasoning steps may not have direct Lean equivalents.

Verification Layer: Lean 4's type checker verifies the proof for *type correctness*. If the Lean code type-checks, the proof is valid (modulo the soundness of Lean's kernel). The verification is *sound* and *deterministic*.

Reliability assessment:

- **Strength**: The verification layer is *formally sound*: if Lean accepts the proof, it is correct. The RL training loop provides a *dense reward signal* (type-checking success/failure) that guides the language model toward valid proofs.

- **Weakness**: The semantic reduction layer is the *primary bottleneck*. Translating informal proof sketches into valid Lean 4 code requires resolving ambiguities and inferring missing information. The language model may produce Lean code that is *syntactically valid but semantically incorrect* (e.g., proving a different theorem than intended).

- **Failure mode**: *Semantic drift*—the Lean proof does not match the intended natural-language argument. The proof may be formally correct but prove a different statement.

Audit conclusion: AlphaProof achieves strong reliability through *tight integration* between generation and verification, but the semantic reduction layer remains a bottleneck. The system exemplifies Pattern 3 (semantic reduction bottleneck).

4.3 FunSearch: Program Search with LLMs

FunSearch [11] uses LLMs to generate programs that are then evaluated by a symbolic executor. It discovered new solutions to the cap set problem and the Erdős-Rado sunflower problem.

Generation Layer: An LLM (based on Codex) generates candidate *Python programs* that implement heuristics for combinatorial search problems. The programs are short (typically <50 lines) and operate on abstract data structures (sets, graphs, functions).

Semantic Reduction Layer: The generated Python code is parsed into an *abstract syntax tree* (AST). The reduction is *well-defined*: Python syntax is unambiguous, and the AST is a faithful representation of the code. However, the reduction does not capture *semantic intent*—the AST does not represent what the program is *supposed* to compute.

Verification Layer: A *symbolic executor* runs the generated programs on test inputs and checks the outputs against known solutions or mathematical properties (e.g., "the output set has no three-term arithmetic progression"). The verification is *executable* but *incomplete*: it only checks correctness on the test inputs, not for all possible inputs.

Reliability assessment:

- **Strength**: The semantic reduction is *exact* (Python → AST). The verification is *pragmatic*: it checks correctness by execution, which is sufficient for many combinatorial problems. The system uses a *function cache* to avoid re-evaluating duplicate programs.
- **Weakness**: The verification is *incomplete*: a program may pass all test cases but fail on unseen inputs. The generation layer may produce programs that are *syntactically valid but semantically meaningless* (e.g., infinite loops, type errors at runtime).
- **Failure mode**: *Verification incompleteness*—the program passes test cases but is incorrect in general. This is mitigated by using a large and diverse set of test inputs.

Audit conclusion: FunSearch achieves reliability through *executable verification* and *iterative refinement* (the LLM is prompted with examples of successful programs). The system exemplifies Pattern 2 (executable verification is pragmatic).

4.4 Ramanujan Machine: Conjecture Generation

The Ramanujan Machine [10] uses AI to search for conjectures about fundamental mathematical constants (e.g., e, π, $\zeta(3)$). It has proposed several conjectures that were later proved by human mathematicians.

Generation Layer: The system uses *machine learning algorithms* (including genetic algorithms and Monte Carlo tree search) to propose conjectures in the form of *continued fractions*, *series expansions*, or *algebraic relations* involving mathematical constants.

Semantic Reduction Layer: Conjectures are encoded as *symbolic expressions* in a computer algebra system (e.g., Mathematica). The reduction is *straightforward*: the conjecture format is restricted (e.g., continued fractions of a specific form), so the mapping from conjecture to symbolic expression is unambiguous.

Verification Layer: The system uses *high-precision numerical computation* (up to thousands of decimal places) to check whether the conjectured relation holds numerically. For example, if the conjecture is $\pi = 3 + \cfrac{1^2}{6+\cfrac{3^2}{6+\cfrac{5^2}{6+\cdots}}}$, the system computes both sides to 10,000 decimal places and checks if they agree.

Reliability assessment:

- **Strength**: The semantic reduction is *exact* (conjecture format is restricted). The numerical verification is *highly reliable*: if a conjecture holds to 10,000 decimal places, it is almost certainly correct (though not formally proved).
- **Weakness**: The verification is *incomplete*: numerical evidence is not a proof. There exist "false" conjectures that hold to many decimal places but are ultimately incorrect (e.g., the Mertens conjecture, which held for all $n < 10^{14}$ but was disproved).

- **Failure mode**: *Numerical coincidence*—the conjecture holds to high precision but is ultimately false. This is mitigated by checking to very high precision and by cross-referencing with known mathematical results.

Audit conclusion: The Ramanujan Machine achieves reliability through *high-precision numerical verification* and *restricted conjecture format*. However, the verification layer is fundamentally *incomplete* (numerical evidence $\neq$ proof). The system exemplifies the tension between *pragmatic reliability* and *formal certainty*.

4.5 AlphaTensor: Discovering Algorithms via Reinforcement Learning

AlphaTensor [7] uses reinforcement learning [13] to discover faster algorithms for fundamental algebraic operations, notably matrix multiplication. It discovered a new algorithm for 4×4 matrix multiplication that uses fewer multiplications than the previous best known algorithm.

Generation Layer: A *reinforcement learning agent* (based on AlphaZero) generates sequences of *bilinear operations* (additions and multiplications of matrix entries). The agent plays a "game" where each move corresponds to choosing a multiplication or addition, and the goal is to express the target tensor (e.g., the matrix multiplication tensor) as a sum of rank-1 tensors.

Semantic Reduction Layer: The generated sequence of operations is encoded as a *tensor decomposition*. The reduction is *exact*: each sequence of operations corresponds to a unique tensor decomposition, and vice versa. The reduction layer uses *group-theoretic symmetries* to reduce the search space (e.g., permutations of matrix indices).

Verification Layer: The resulting algorithm is verified by *exhaustive testing*: the algorithm is run on all possible inputs (for small matrices) or a large random sample (for larger matrices), and the outputs are compared against the known correct result. The verification is *executable* and *highly reliable* (though not formally proved for all inputs).

Reliability assessment:

- **Strength**: The semantic reduction is *exact* (operations $\leftrightarrow$ tensor decomposition). The verification is *exhaustive* for small cases. The RL agent uses *Monte Carlo tree search* to explore the space of algorithms efficiently.
- **Weakness**: The verification is *incomplete* for large cases (only a random sample is tested). The generation layer may produce algorithms that are *correct but inefficient* (e.g., using more operations than necessary).
- **Failure mode**: *Verification incompleteness*—the algorithm passes the test cases but is incorrect for some unseen inputs. This is mitigated by testing on a large and diverse set of inputs.

Audit conclusion: AlphaTensor achieves reliability through *exact semantic reduction* and *exhaustive verification* for small cases. The system exemplifies Pattern 1 (domain restriction improves reliability) and Pattern 2 (executable verification is pragmatic).

5 Cross-System Analysis: Patterns and Insights

Table 1 summarizes our analysis of the five systems.

5.1 Pattern 1: Domain Restriction Improves Reliability

Systems that restrict the generation layer to a narrow domain (AlphaGeometry, AlphaTensor) achieve higher reliability than general-purpose systems. This is because:

- The semantic reduction layer is simpler and more well-defined.
- The verification layer can be exhaustive within the restricted domain.

Table 1: Comparison of AI4Math systems across the three-layer framework

System	Generation	Semantic Reduction	Verification
AlphaGeometry	Neural (narrow)	Symbolic constraints	Exhaustive search
AlphaProof	LLM + RL	Lean 4 code	Formal proof
FunSearch	LLM	Python AST	Execution
Ramanujan	AI search	Symbolic expressions	Numerical
AlphaTensor	RL	Tensor operations	Execution

5.2 Pattern 2: Executable Verification is Pragmatic

Systems that use executable verification (FunSearch, AlphaTensor) achieve practical reliability without formal proof. This approach is limited to problems where correctness can be checked by running the program, but it covers many important cases.

5.3 Pattern 3: The Semantic Reduction Bottleneck

Across all systems, the semantic reduction layer is the primary source of failure. When the mapping from natural language to structured representation is ambiguous or lossy, errors propagate to the verification layer.

5.4 Pattern 4: Bidirectional Consistency is Necessary but Insufficient

We define *bidirectional consistency* as the property that:

1. The structured representation faithfully encodes the natural-language reasoning (*forward consistency*).
2. The natural-language reasoning can be recovered from the structured representation (*backward consistency*).

Bidirectional consistency is *necessary* for reliability: if the structured representation does not match the intended meaning, verification is meaningless. However, it is *not sufficient*: the structured representation itself may be incorrect (e.g., due to errors in the generation layer).

6 An Operational Reliability Checklist

Based on our analysis, we propose the following checklist for practitioners developing AI4Math systems:

7 Discussion

7.1 Limitations

Our framework has several limitations:

- **Design-level analysis**: We analyze existing systems at the design level, without empirical evaluation of reliability metrics.
- **Limited case studies**: We focus on five representative systems; other AI4Math approaches may exhibit different patterns.
- **Informal verification**: Our checklist is a practical guide, not a formal verification method.

Table 2: Reliability checklist for AI4Math systems

Generation Layer
☐ Is the generation domain explicitly bounded?
☐ Are hallucination detection mechanisms in place?
☐ Is there a mechanism for detecting logical gaps?
Semantic Reduction Layer
☐ Is the target IR well-defined and unambiguous?
☐ Is the mapping from natural language to IR documented?
☐ Are bidirectional consistency checks implemented?
☐ Is type information preserved or inferred correctly?
Verification Layer
☐ Is the verification method sound (no false positives)?
☐ Are verification failures handled gracefully?
☐ Is the verification process reproducible?
☐ Are the limitations of verification explicitly documented?

7.2 Future Work

Several directions merit further investigation:

- **Empirical validation**: Applying the framework to measure reliability across a broader range of AI4Math systems.
- **Formalization**: Formalizing the semantic reduction layer using type theory or category theory.
- **Automation**: Developing tools that automatically check bidirectional consistency.
- **Extension to other domains**: Applying the framework to AI-assisted reasoning in physics, chemistry, or other formal sciences.

8 Conclusion

We have proposed a three-layer audit framework for assessing the reliability of AI-assisted mathematical reasoning. Our framework decomposes the reasoning process into Generation, Semantic Reduction, and Verification layers, each with distinct responsibilities and failure modes.

Through case studies of five representative AI4Math systems, we have identified four key patterns:

1. Domain restriction improves reliability by simplifying the semantic reduction layer.
2. Executable verification is a pragmatic alternative to formal proof.
3. The semantic reduction layer is the primary bottleneck for end-to-end reliability.
4. Bidirectional consistency is necessary but insufficient for trustworthy AI mathematics.

We have also proposed an operational reliability checklist for practitioners. Our analysis reveals that reliable AI mathematics requires not just better generation or stronger verification, but a *structured semantic layer* that faithfully bridges the gap between informal reasoning and formal checking.

Acknowledgments

We thank the reviewers for their constructive feedback. This work was supported by [funding information].

References

[1] Xiaoyang Chen and Xiang Jiang. Lectures on ai for mathematics. 2026. Internal lecture notes, version 1.0, 2026-04-11.

[2] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019. Verified via arXiv:1911.01547.

[3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. Verified via arXiv:2110.14168 (OpenAI GSM8K).

[4] Alex Davies, Petar Veličković, Nenad Buesing, et al. Advancing mathematics by guiding human intuition with ai. *Nature*, 600(7887):70–74, 2021. Verified via Nature website.

[5] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008. Verified via Springer LNCS.

[6] Leonardo de Moura, Soonho Kong, Jeremy Avigad, et al. The lean theorem prover (system description). *Journal of Automated Reasoning*, 55(4):337–362, 2015. Verified via Springer.

[7] Alhussein Fawzi, Matej Balog, Aja Huang, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022. Verified via Nature website (AlphaTensor).

[8] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. Verified via arXiv:2211.10435.

[9] Haocheng Ju and Bin Dong. Ai for mathematics: Progress, challenges, and prospects. *arXiv preprint arXiv:2601.13209*, 2025. Verified via arXiv:2601.13209.

[10] Gal Raayoni, Shahar Gottlieb, Yahav Manor, et al. Generating conjectures on fundamental constants with the ramanujan machine. *Nature*, 590(7844):67–73, 2021. Verified via Nature website.

[11] Bernardino Romera-Paredes, Mateja Baresch, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. Verified via Nature website (FunSearch).

[12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023. Verified via arXiv:2302.04761 and NeurIPS 2023 proceedings.

[13] Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT press, 2nd edition, 2018. Verified via MIT Press catalog.

[14] Trieu H Trinh, Yuhuai Wu, Quoc Le, Xinyun He, and Xinyun Lu. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. Verified via Nature website (AlphaGeometry).

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017. Verified via NeurIPS proceedings.

[16] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022. Verified via arXiv:2201.11903 and NeurIPS 2022 proceedings.